

Validaciones Básicas

Protegiendo Nuestros Formularios

Python Full Stack - Clase 30

Ejemplos que Usaremos

En esta clase trabajaremos con los ejemplos existentes:

- `00_tacos_relaciones_mucho_a_muchos` : Para validaciones básicas
- `02_modularizacion_crud_usuario` : Para validaciones avanzadas con regex y unicidad

Nota: Estos ejemplos ya tienen las validaciones implementadas. Las revisaremos paso a paso.

¿Qué aprenderemos hoy?

-  Entender por qué validamos la información del usuario
-  Crear métodos de validación en nuestros modelos
-  Implementar mensajes flash para mostrar errores
-  Validar campos obligatorios y longitud mínima
-  Usar expresiones regulares para validar patrones
-  Mostrar mensajes de error en las plantillas
-  Redirigir cuando hay errores de validación
-  Practicar con ejercicios reales

¿Por Qué Validar?

El Problema que Resolvemos

¿Notaste en ejercicios pasados que si enviabas un formulario **vacío**, se guardaba un nuevo registro sin información?

¡Esto no debería ser posible! 

Usuario envía formulario vacío



¿Qué pasa? Se guarda en BD sin datos



 Problema: Datos inválidos en la base de datos

¿Qué son las Validaciones?

Las **validaciones** son reglas que verifican que el usuario ingrese la información que nosotros estamos esperando.

Piensa en ellas como un "guardián"  que revisa todo antes de permitir que se guarde.

Reglas de Validación Comunes

Las validaciones más comunes incluyen:

-  Comprobar que los datos estén presentes (no vacíos)
-  Asegurarse que los datos tengan el formato correcto (email, teléfono, etc.)
-  Verificar longitud mínima/máxima (contraseñas, nombres)
-  Validar valores únicos (emails, usuarios)
-  Comprobar rangos (edades, precios)

El Flujo de Validación

Usuario llena formulario



¿Datos válidos?



NO

SÍ



Mostrar
errores

Guardar
en BD

Cada validación es una condicional:

- SI algo no es válido → avisamos al usuario
- **DE LO CONTRARIO** → guardamos en la base de datos

Validación Básica de Formulario

Paso 1: Método de Validación en el Modelo

Crearemos un `@staticmethod` en nuestro modelo para validar los datos.

¿Recuerdas qué es un método estático?

- Son funciones ayudantes o auxiliares
- No tienen acceso a atributos de instancia ni de clase
- La información a validar la recibimos como argumento

Estructura del Método de Validación

```
from flask import flash # Importamos flash para mensajes

class Taco:
    # ... código anterior (atributos, métodos, etc.) ...

    @staticmethod
    def validar_taco(data):
        es_valido = True # Asumimos que es válido

        # Validaciones aquí...

        return es_valido # Retornamos True o False
```

- `@staticmethod`: No necesita instancia ni clase
- Recibe un diccionario con los datos del formulario
- Retorna `True` si es válido, `False` si no

```
@staticmethod
def validar_taco(data):
    es_valido = True # Asumimos que es válido

    # Si la tortilla tiene menos de 3 caracteres
    if len(data['tortilla']) < 3:
        flash("La tortilla debe tener al menos 3 caracteres")
        es_valido = False # Ya no es válido

    if len(data['guiso']) < 3:
        flash("El guiso debe tener al menos 3 caracteres")
        es_valido = False

    if len(data['salsa']) < 3:
        flash("La salsa debe tener al menos 3 caracteres")
        es_valido = False

    return es_valido
```

¿Qué hace cada línea?

- `len()` : Cuenta los caracteres
- `flash()` : Crea un mensaje temporal de error

¿Qué son los Mensajes Flash?

Flash usa la sesión para mostrar mensajes temporales en la página.

Características:

-  Son temporales (desaparecen después de una redirección)
-  Perfectos para mensajes de error
-  Se muestran una sola vez
-  Requieren configuración de clave secreta

Diferencia con sesión:

- Sesión: Se mantiene hasta eliminarla manualmente
- Flash: Desaparece automáticamente después de mostrarse

Configuración Necesaria

Para usar mensajes flash, necesitamos configurar la **clave secreta** en `__init__.py`:

```
from flask import Flask

app = Flask(__name__)
app.secret_key = "tu_clave_secreta_aqui" # ¡IMPORTANTE!
```

¿Por qué? Flask usa la sesión para almacenar los mensajes flash, y la sesión requiere una clave secreta para seguridad.

Ya tenemos el método de validación. Ahora lo invocamos **antes** de guardar:

```
@app.route('/crear', methods=['POST'])
def crear():
    datos = {
        "tortilla": request.form['tortilla'],
        "guiso": request.form['guiso'],
        "salsa": request.form['salsa'],
        "restaurante_id": request.form['restaurante_id']
    }

    # Llamamos al método para validar
    if not Taco.validar_taco(datos):
        return redirect('/') # Si NO es válido, redirigimos

    # Si es válido, guardamos
    Taco.save(datos)
    return redirect('/tacos')
```

Flujo:

1. Recibimos los datos del formulario
2. Validamos con `validar_taco()`

Paso 3: Mostrar Mensajes Flash en la Plantilla

Los mensajes flash se muestran en la plantilla HTML usando Jinja2:

```
{% with messages = get_flashed_messages() %}
  {% if messages %}
    {% for message in messages %}
      <p class="error">{{ message }}</p>
    {% endfor %}
  {% endif %}
{% endwith %}
```

- `get_flashed_messages()` : Obtiene todos los mensajes flash
- `{% if messages %}` : Verifica si hay mensajes
- `{% for message in messages %}` : Recorre cada mensaje
- `{{ message }}` : Muestra el mensaje

Ejemplo Completo: Formulario con Validación

```
<form action="/crear" method="POST">
  <!-- Mensajes de error -->
  {% with messages = get_flashed_messages() %}
    {% if messages %}
      <div class="errores">
        {% for message in messages %}
          <p>{{ message }}</p>
        {% endfor %}
      </div>
    {% endif %}
  {% endwith %}

  <!-- Campos del formulario -->
  <input type="text" name="tortilla" placeholder="Tipo de tortilla">
  <input type="text" name="guiso" placeholder="Guiso">
  <input type="text" name="salsa" placeholder="Salsa">
  <button type="submit">Crear Taco</button>
</form>
```

Patrones de Validación

¿Qué son las Expresiones Regulares?

Habrán ocasiones en las que debemos revisar si el texto tiene un **patrón específico**.

Ejemplos de patrones:

-  Email: debe tener "@" y un punto
-  Teléfono: formato específico de números
-  Contraseña: debe tener mayúsculas, números, etc.

No podemos hacer comparación literal → Necesitamos expresiones regulares.

Expresiones Regulares (Regex)

Las **expresiones regulares**, abreviadas como **regex**, son herramientas poderosas para encontrar patrones en cadenas de texto.

Características:

-  Definen patrones específicos
-  Validan formatos (emails, teléfonos, etc.)
-  Imponen requisitos específicos (contraseñas)
-  Ofrecen validación flexible y eficiente

Por ahora: No necesitas dominar la creación de regex, pero sí entender cómo usarlas.

Validar Email con Regex

```
r'^[a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z]+$'
```

¿Qué significa?

- `^` : Inicio de la cadena
- `[a-zA-Z0-9._-]+` : Letras, números y algunos caracteres especiales
- `@` : Debe tener un arroba
- `[a-zA-Z0-9._-]+` : Dominio antes del punto
- `\.` : Punto literal (escapado)
- `[a-zA-Z]+` : Extensión del dominio
- `$` : Fin de la cadena

La `r` al inicio indica que es una cadena "cruda" (raw string).

```

import re # Importamos el módulo de expresiones regulares

# Objeto de expresión regular para validar email
EMAIL_REGEX = re.compile(r'^[a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z]+$')

class Usuario:
    # ... código anterior ...

    @staticmethod
    def validar_usuario(data):
        es_valido = True

        # Validar que el email no esté vacío
        if len(data['email'].strip()) == 0:
            flash("El email es obligatorio", "email")
            es_valido = False

        # Revisa si el email coincide con el patrón
        elif not EMAIL_REGEX.match(data['email']):
            flash("E-mail inválido. Debe tener formato válido (ejemplo@dominio.com)", "email")
            es_valido = False

        return es_valido

```

¿Qué hace `.match()` ?

- Retorna `None` si NO encuentra coincidencia

```

@app.route("/usuarios/crear", methods=["POST"])
def crear_usuario():
    # Guardar datos en sesión antes de validar
    session['form_data'] = request.form.to_dict()

    data = {
        "nombre": request.form["nombre"],
        "apellido": request.form["apellido"],
        "email": request.form["email"],
        "edad": request.form["edad"]
    }

    # Validar antes de guardar
    if not Usuario.validar_usuario(data):
        return redirect("/usuarios/nuevo") # Si NO es válido, redirigimos

    # Si es válido, limpiar sesión y guardar
    session.pop('form_data', None)
    Usuario.save(datos=data)
    return redirect("/usuarios")

```

Flujo:

1. Usuario envía formulario
2. Guardamos datos en sesión (por si hay errores)
3. Validamos con regex y otras reglas

Mensajes Flash con Categorías

A veces queremos **categorizar los mensajes** para mostrarlos en lugares distintos.

```
flash("El correo es obligatorio", "correo") # Categoría "correo"  
flash("La contraseña es muy corta", "password") # Categoría "password"
```

Mostrar mensajes por categoría:

```
{% with messages = get_flashed_messages(category_filter=["correo"]) %}  
  {% if messages %}  
    {% for message in messages %}  
      <p class="error-correo">{{ message }}</p>  
    {% endfor %}  
  {% endif %}  
{% endwith %}
```


Más Herramientas de Validación

Métodos de cadena útiles:

```
# Solo letras
if not texto.isalpha():
    flash("Solo se permiten letras")

# Solo números
if not texto.isdigit():
    flash("Solo se permiten números")

# Letras y números
if not texto.isalnum():
    flash("Solo letras y números permitidos")
```

Validar fechas:

```
from datetime import datetime

try:
    fecha = datetime.strptime(fecha_str, "%Y-%m-%d")
except ValueError:
    flash("Fecha inválida")
```



Validación de Email con Base de Datos

Práctica: Validación de Email

Vamos a trabajar con el proyecto `02_modularizacion_crud_usuario` que ya tenemos.

Objetivos:

-  Agregar validaciones para la creación de usuarios
-  Validar formato de email con regex
-  Validar que el email sea único en la base de datos
-  Mostrar mensajes de error apropiados
-  Mantener datos del formulario cuando hay errores

Validaciones Requeridas

Método `validar_usuario()` debe verificar:

1.  Campos no vacíos: Nombre, apellido, email
2.  Formato de email: Usar regex para validar formato
3.  Email único: Verificar que no exista en la base de datos
4.  Mensajes flash: Mostrar errores específicos
5.  Redirección: Si hay errores, volver al formulario

Validación de Campos Vacíos

```
@staticmethod
def validar_usuario(data):
    es_valido = True

    # Validar que el nombre no esté vacío
    if len(data['nombre'].strip()) == 0:
        flash("El nombre es obligatorio")
        es_valido = False

    # Validar que el apellido no esté vacío
    if len(data['apellido'].strip()) == 0:
        flash("El apellido es obligatorio")
        es_valido = False

    # Validar que el email no esté vacío
    if len(data['email'].strip()) == 0:
        flash("El email es obligatorio")
        es_valido = False

    return es_valido
```

.strip() : Elimina espacios al inicio y final.

Validación de Formato de Email

```
import re

EMAIL_REGEX = re.compile(r'^[a-zA-Z0-9.+_-]+@[a-zA-Z0-9._-]+\.[a-zA-Z]+$')

@staticmethod
def validar_usuario(data):
    es_valido = True

    # ... validaciones anteriores ...

    # Validar formato de email
    if not EMAIL_REGEX.match(data['email']):
        flash("E-mail inválido")
        es_valido = False

    return es_valido
```

Validación de Email Único

```
@classmethod
def buscar_por_email(cls, email):
    query = "SELECT * FROM usuarios WHERE email = %(email)s"
    resultado = connectToMySQL('tienda').query_db(query, {'email': email})

    if len(resultado) > 0:
        return cls(resultado[0]) # Retorna el usuario encontrado
    return None # No existe

@staticmethod
def validar_usuario(data):
    es_valido = True

    # ... validaciones anteriores (campos obligatorios, formato email) ...

    # Validar que el email sea único en la base de datos
    elif Usuario.buscar_por_email(data['email']):
        flash("Este email ya está registrado. Por favor use otro email.", "email")
        es_valido = False

    return es_valido
```


Nota: Usamos `elif` para evitar consultar la BD si el email ya falló otras validaciones.

BONUS: Mantener Datos del Formulario

```
@app.route("/usuarios/crear", methods=["POST"])
def crear_usuario():
    # Guardamos los datos en sesión antes de validar
    session['form_data'] = request.form.to_dict()

    data = {
        "nombre": request.form["nombre"],
        "apellido": request.form["apellido"],
        "email": request.form["email"],
        "edad": request.form["edad"]
    }

    if not Usuario.validar_usuario(data):
        return redirect("/usuarios/nuevo") # Los datos están en sesión

    # Si es válido, limpiamos la sesión
    session.pop('form_data', None)
    Usuario.save(datos=data)
    return redirect("/usuarios")
```

En la plantilla `nuevo_usuario.html` :

```
<input type="text" name="nombre"
      value="{{ form_data.nombre if form_data else '' }}" required>
<input type="text" name="apellido"
      value="{{ form_data.apellido if form_data else '' }}" required>
<input type="email" name="email"
      value="{{ form_data.email if form_data else '' }}" required>
```

Ventaja: El usuario no pierde los datos ingresados cuando hay errores.

Estructura Completa del Método

```
@staticmethod
def validar_usuario(data):
    es_valido = True

    # Validar que el nombre no esté vacío
    if len(data['nombre'].strip()) == 0:
        flash("El nombre es obligatorio", "nombre")
        es_valido = False

    # Validar que el nombre tenga al menos 2 caracteres
    elif len(data['nombre'].strip()) < 2:
        flash("El nombre debe tener al menos 2 caracteres", "nombre")
        es_valido = False

    # Validar que el apellido no esté vacío
    if len(data['apellido'].strip()) == 0:
        flash("El apellido es obligatorio", "apellido")
        es_valido = False

    # Validar que el apellido tenga al menos 2 caracteres
    elif len(data['apellido'].strip()) < 2:
        flash("El apellido debe tener al menos 2 caracteres", "apellido")
        es_valido = False

    # Validar que el email no esté vacío
    if len(data['email'].strip()) == 0:
        flash("El email es obligatorio", "email")
        es_valido = False

    # Validar formato de email con regex
    elif not EMAIL_REGEX.match(data['email']):
        flash("E-mail inválido. Debe tener formato válido (ejemplo@dominio.com)", "email")
        es_valido = False

    # Validar que el email sea único en la base de datos
    elif Usuario.buscar_por_email(data['email']):
        flash("Este email ya está registrado. Por favor use otro email.", "email")
        es_valido = False

    return es_valido
```

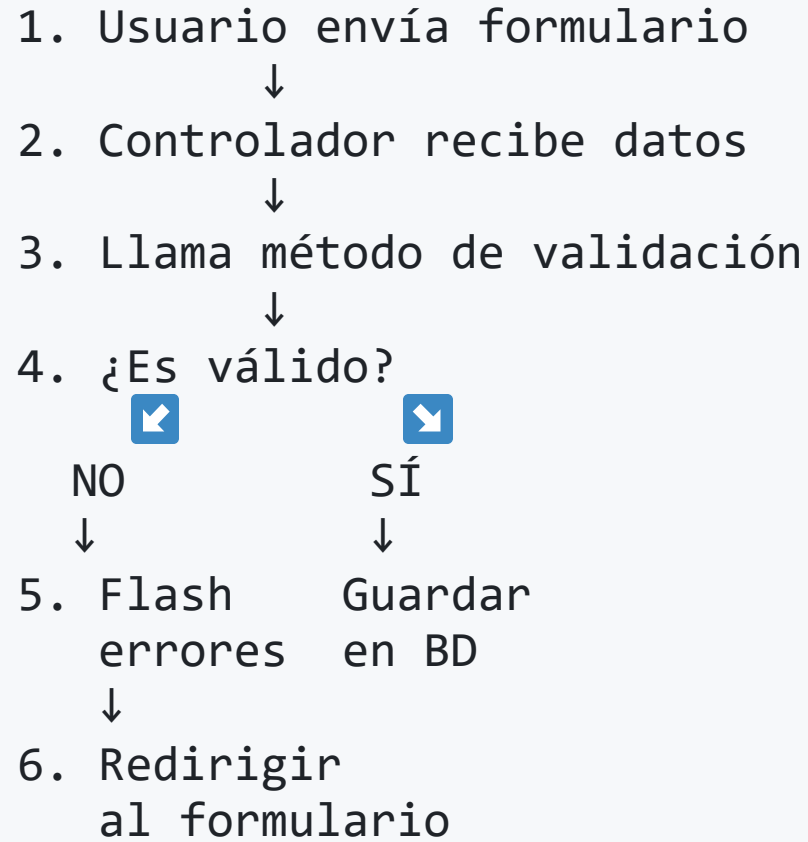


Resumen de Conceptos Clave

Lo Más Importante

1. **Validaciones** protegen nuestros datos de información inválida
2. **Métodos estáticos** (`@staticmethod`) son perfectos para validaciones
3. **Mensajes flash** muestran errores temporales al usuario
4. **Expresiones regulares** validan patrones complejos (emails, teléfonos)
5. **Validación de unicidad** requiere consultar la base de datos
6. **Redirección** después de errores mejora la experiencia del usuario

Flujo Completo de Validación



Conceptos Técnicos

Métodos estáticos:

- `@staticmethod` : No necesita instancia ni clase
- Perfectos para funciones auxiliares como validaciones

Mensajes flash:

- `flash("mensaje")` : Crea mensaje temporal
- `get_flashed_messages()` : Obtiene mensajes en plantilla
- Requiere `secret_key` configurada

Expresiones regulares:

- `re.compile()` : Crea patrón regex
- `.match()` : Verifica coincidencia
- Retorna `None` si no coincide

Buenas Prácticas

- ✓ Validar siempre antes de guardar en base de datos
- ✓ Mensajes claros que expliquen qué está mal
- ✓ Validar en el servidor (nunca confiar solo en validación del cliente)
- ✓ Usar métodos estáticos para validaciones reutilizables
- ✓ Redirigir después de errores para mejor UX
- ✓ Mantener datos del formulario cuando sea posible

Errores Comunes a Evitar

- ✗ Olvidar configurar `secret_key` → Los mensajes flash no funcionan
- ✗ No validar campos vacíos → Se pueden guardar datos inválidos
- ✗ Validar solo en el cliente → Los hackers pueden saltarse validaciones
- ✗ Mensajes de error confusos → El usuario no sabe qué corregir
- ✗ No redirigir después de errores → El usuario ve página en blanco

Recursos para Seguir Aprendiendo

Documentación Oficial

-  [Flask - Message Flashing](#)
-  [Python - re module](#)
-  [Python String Methods](#)
-  [Regex101](#) - Herramienta interactiva para probar regex

Próximos Pasos

En la siguiente clase aprenderemos:

-  Protección de contraseñas con hashing
-  Bcrypt para seguridad avanzada
-  Sistema de autenticación completo
-  Login y registro de usuarios

Práctica para Casa

Proyecto sugerido:

Crea una aplicación de **gestión de tareas** con validaciones completas:

1. Validar que el título tenga al menos 3 caracteres
2. Validar que la descripción tenga al menos 10 caracteres
3. Validar formato de fecha
4. Validar que la prioridad sea válida (alta, media, baja)
5. Mostrar mensajes de error con categorías
6. Mantener datos del formulario cuando hay errores

 ¡Felicidades!

Ya sabes validar formularios

Has aprendido a proteger tus aplicaciones de datos inválidos 

? Preguntas

¿Alguna duda sobre validaciones?

 ¡Excelente Trabajo!

¡Nos vemos en la siguiente clase con autenticación!

No olvides completar todos los ejercicios 